In [9]:
```
import os
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
import wfdb
import ast
from tqdm import tqdm
from sklearn.preprocessing import MultiLabelBinarizer
from sklearn.metrics import classification_report, roc_auc_score, average_precision
from sklearn.metrics import f1_score
import matplotlib.pyplot as plt
```

In [10]:
```
DATA_PATH = "C:\\Users\\Me\\Desktop\\ECG\\data\\"
SAMPLING_RATE = 100
BATCH_SIZE = 64
EPOCHS = 10

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
device
```

Out[10]:
```
device(type='cuda')
```

In [11]:
```
Y = pd.read_csv(DATA_PATH + "ptbxl_database.csv", index_col="ecg_id")
Y.scp_codes = Y.scp_codes.apply(lambda x: ast.literal_eval(x))
```

In [12]:
```
agg_df = pd.read_csv(DATA_PATH + "scp_statements.csv", index_col=0)
agg_df = agg_df[agg_df.diagnostic == 1]

def aggregate_diagnostic(y_dic):
    tmp = []
    for key in y_dic.keys():
        if key in agg_df.index:
            tmp.append(agg_df.loc[key].diagnostic_class)
    return list(set(tmp))

Y["diagnostic_superclass"] = Y.scp_codes.apply(aggregate_diagnostic)
```

In [13]:
```
CACHE_FILE = os.path.join(DATA_PATH, "X.npy")

def load_raw_data(df, sampling_rate, path, cache_file):
    if os.path.exists(cache_file):
        print("Loading preprocessed data from cache...")
        X = np.load(cache_file, mmap_mode='r')
        print("Done. Shape:", X.shape)
        return X

    if sampling_rate == 100:
        files = df.filename_lr
    else:
        files = df.filename_hr
```

```
    data = []
    for f in tqdm(files, desc="Loading ECG signals"):
        signal, _ = wfdb.rdsamp(os.path.join(path, f))
        data.append(signal)

    X = np.array(data)

    np.save(cache_file, X)
    print(f"Saved cache to {cache_file}")

    return X

X = load_raw_data(Y, SAMPLING_RATE, DATA_PATH, CACHE_FILE)
```

```
Loading preprocessed data from cache...
Done. Shape: (21799, 1000, 12)
```

In [14]:

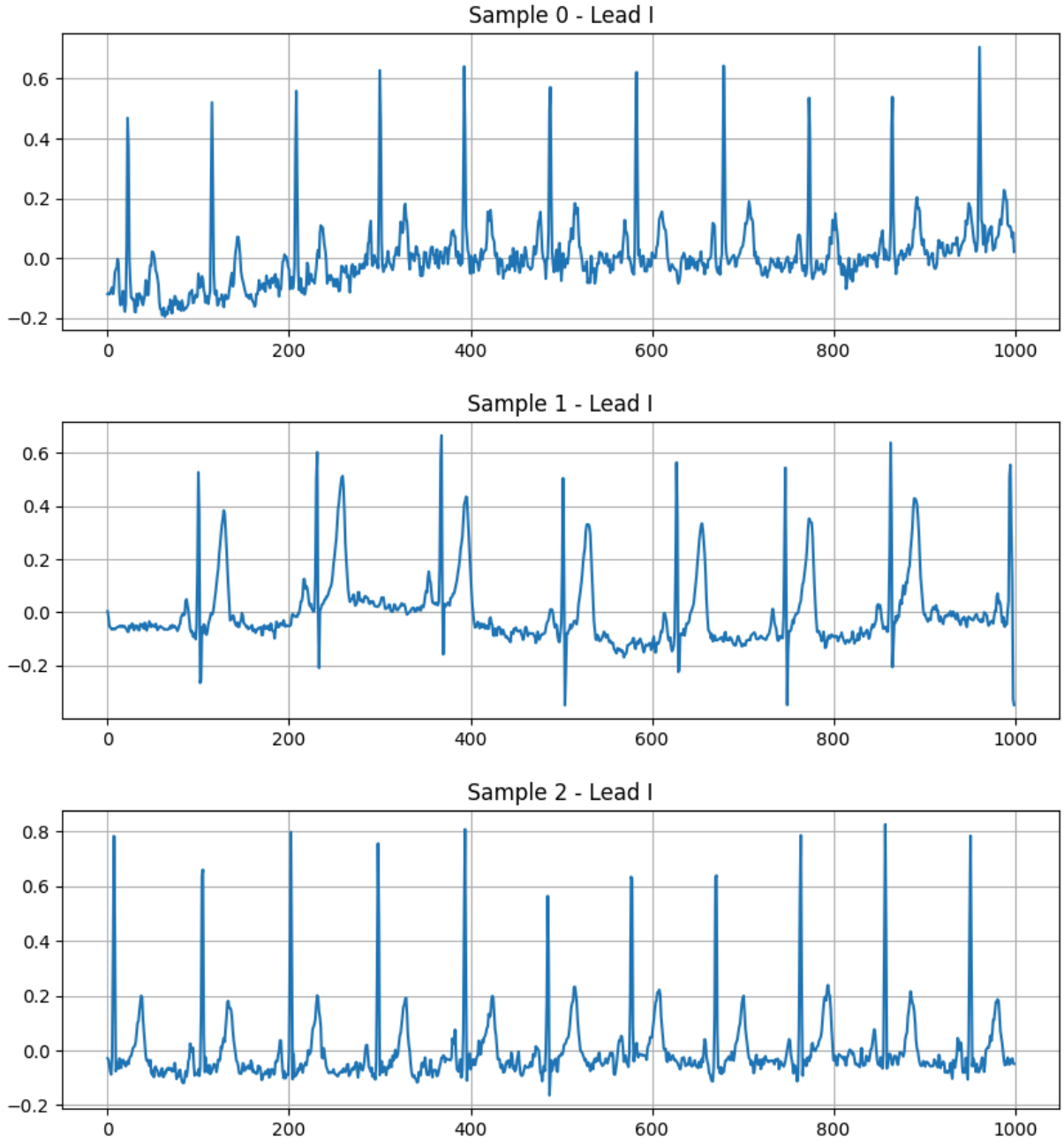
```
print("===== DATA OVERVIEW =====")
print("Shape:", X.shape)
print("Min:", np.min(X))
print("Max:", np.max(X))
print("Mean:", np.mean(X))
print("Std:", np.std(X))

print("\nNaN count:", np.isnan(X).sum())
```

```
===== DATA OVERVIEW =====
Shape: (21799, 1000, 12)
Min: -20.032
Max: 17.212
Mean: -0.0007808800556600472
Std: 0.23539237334110466

NaN count: 0
```

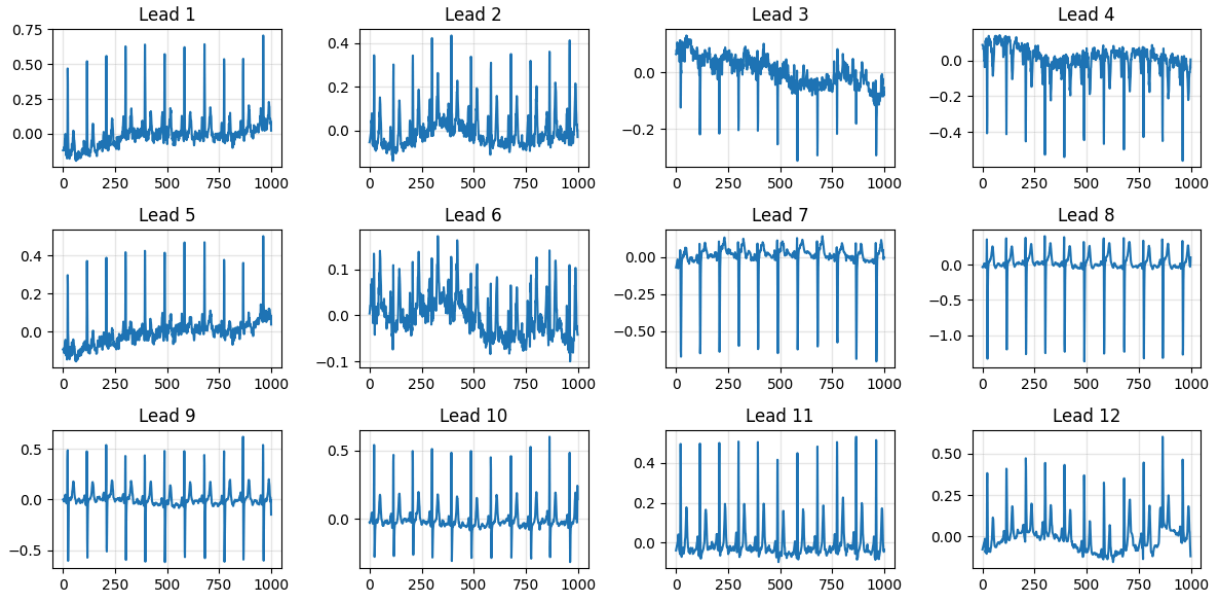
In [15]:

```
print("\n===== SAMPLE SIGNALS =====")

for i in range(3):
    plt.figure(figsize=(10,3))
    plt.plot(X[i][:, 0])  # Lead I
    plt.title(f"Sample {i} - Lead I")
    plt.grid(True)
    plt.show()
```

```
===== SAMPLE SIGNALS =====
```



In [16]:
```
sample = X[0]

fig, axes = plt.subplots(3, 4, figsize=(12,6))

for i, ax in enumerate(axes.flatten()):
    ax.plot(sample[:, i])
    ax.set_title(f"Lead {i+1}")
    ax.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()
```



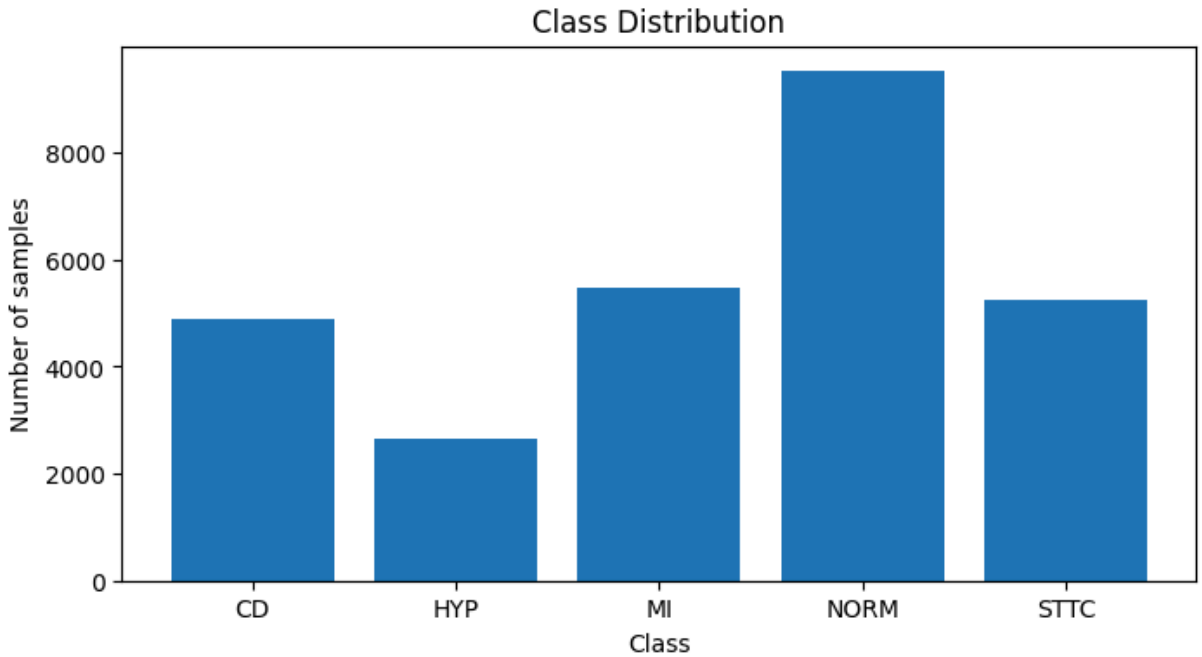
In [17]:
```
mlb = MultiLabelBinarizer()
y = mlb.fit_transform(Y["diagnostic_superclass"])

class_names = mlb.classes_
class_names
```

Out[17]:
```
array(['CD', 'HYP', 'MI', 'NORM', 'STTC'], dtype=object)
```

In [18]:
```
print("===== CLASS DISTRIBUTION =====")

class_counts = y.sum(axis=0)

for cls, count in zip(class_names, class_counts):
    print(f"{cls}: {count}")
```

```
===== CLASS DISTRIBUTION =====
CD: 4898
HYP: 2649
MI: 5469
NORM: 9514
STTC: 5235
```

In [19]:
```
plt.figure(figsize=(8,4))
plt.bar(class_names, class_counts)
plt.title("Class Distribution")
plt.xlabel("Class")
plt.ylabel("Number of samples")
plt.show()
```



In [20]:
```
test_fold = 10

train_idx = np.where(Y.strat_fold != test_fold)[0]
test_idx  = np.where(Y.strat_fold == test_fold)[0]

X_train, X_test = X[train_idx], X[test_idx]
y_train, y_test = y[train_idx], y[test_idx]

X_train.shape, X_test.shape
```

Out[20]:
```
((19601, 1000, 12), (2198, 1000, 12))
```

In [21]:
```
class_counts = y_train.sum(axis=0)
total_samples = y_train.shape[0]

pos_weight = (total_samples - class_counts) / class_counts

pos_weight = torch.tensor(pos_weight, dtype=torch.float32).to(device)

print("pos_weight:", pos_weight)
```

```
pos_weight: tensor([3.4527, 7.2116, 2.9848, 1.2922, 3.1580], device='cuda:0')
```

In [22]:
```
def normalize(signal):
    return (signal - np.mean(signal)) / (np.std(signal) + 1e-8)

print("Normalizing dataset...")
X = np.array([normalize(x) for x in X])
```

```
Normalizing dataset...
```

In [23]:
```
class ECGDataset(Dataset):
    def __init__(self, X, y):
        self.X = torch.tensor(X, dtype=torch.float32)
        self.y = torch.tensor(y, dtype=torch.float32)
```

```
    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx]
```

In [120…

```
from torch.utils.data import WeightedRandomSampler

class_counts = y_train.sum(axis=0)
class_weights = 1.0 / class_counts

sample_weights = (y_train * class_weights).sum(axis=1)

sampler = WeightedRandomSampler(sample_weights, len(sample_weights))

train_loader = DataLoader(
    ECGDataset(X_train, y_train),
    batch_size=64,
    sampler=sampler,
    num_workers=0,
    pin_memory=True
)

test_loader = DataLoader(
    ECGDataset(X_test, y_test),
    batch_size=64,
    shuffle=False,
    num_workers=0,
    pin_memory=True
)
```

In [121…

```
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super().__init__()

        self.conv1 = nn.Conv1d(in_channels, out_channels, kernel_size=3, padding=1)
        self.bn1 = nn.BatchNorm1d(out_channels)
        self.relu = nn.ReLU()

        self.conv2 = nn.Conv1d(out_channels, out_channels, kernel_size=3, padding=1
        self.bn2 = nn.BatchNorm1d(out_channels)

        # if channels change, adjust shortcut
        self.shortcut = nn.Sequential()
        if in_channels != out_channels:
            self.shortcut = nn.Sequential(
                nn.Conv1d(in_channels, out_channels, kernel_size=1),
                nn.BatchNorm1d(out_channels)
            )

    def forward(self, x):
        identity = self.shortcut(x)
```

```
        out = self.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))

        out += identity
        out = self.relu(out)

        return out
```

In [122…

```
class ECGNet(nn.Module):
    def __init__(self, n_classes):
        super().__init__()

        self.block1 = ResidualBlock(12, 64)
        self.block2 = ResidualBlock(64, 128)
        self.block3 = ResidualBlock(128, 256)

        self.pool = nn.AdaptiveAvgPool1d(1)

        self.classifier = nn.Sequential(
            nn.Linear(256, 128),
            nn.ReLU(),
            nn.Dropout(0.3),
            nn.Linear(128, n_classes)
        )

    def forward(self, x):

        x = self.block1(x)
        x = self.block2(x)
        x = self.block3(x)

        x = self.pool(x).squeeze(-1)

        x = self.classifier(x)
        return x
```

In [123…

```
class FocalLoss(nn.Module):
    def __init__(self, pos_weight=None, gamma=2):
        super().__init__()
        self.pos_weight = pos_weight
        self.gamma = gamma

    def forward(self, logits, targets):
        bce = nn.functional.binary_cross_entropy_with_logits(
            logits, targets, pos_weight=self.pos_weight, reduction='none'
        )
        pt = torch.exp(-bce)
        focal = (1 - pt) ** self.gamma * bce
        return focal.mean()
```

In [124…

```
model = ECGNet(len(class_names)).to(device)

criterion = FocalLoss(pos_weight=pos_weight, gamma=2)
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=2)
```

In [125…

```
def train_one_epoch(model, loader):
    model.train()
    total_loss = 0

    for x, y in loader:
        x = x.permute(0, 2, 1).to(device)
        y = y.to(device)

        optimizer.zero_grad()
        logits = model(x)
        loss = criterion(logits, y)

        loss.backward()
        optimizer.step()

        total_loss += loss.item() * x.size(0)

    return total_loss / len(loader.dataset)


def evaluate(model, loader):
    model.eval()
    total_loss = 0

    all_logits = []
    all_targets = []

    with torch.no_grad():
        for x, y in loader:
            x = x.permute(0, 2, 1).to(device)
            y = y.to(device)

            logits = model(x)
            loss = criterion(logits, y)

            total_loss += loss.item() * x.size(0)

            all_logits.append(logits.cpu())
            all_targets.append(y.cpu())

    return (
        total_loss / len(loader.dataset),
        torch.cat(all_logits),
        torch.cat(all_targets)
    )
```

In [126…

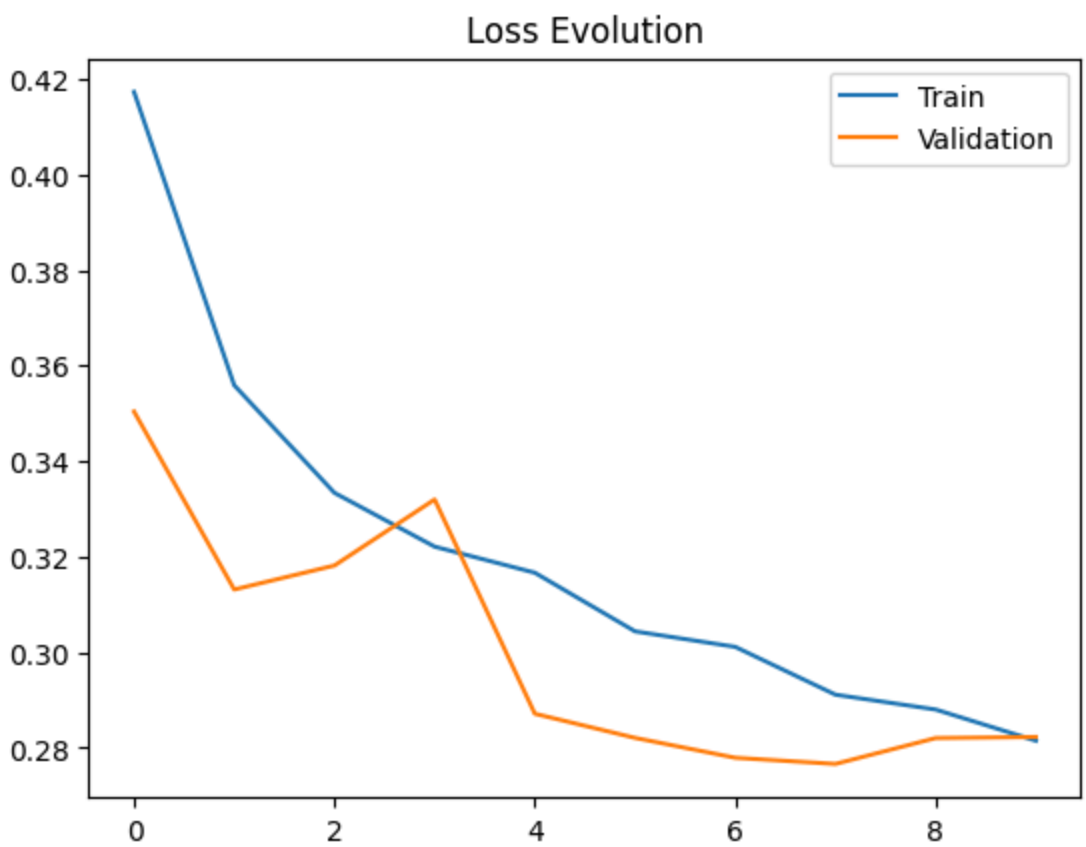
```
train_losses = []
val_losses = []

for epoch in range(EPOCHS):
    train_loss = train_one_epoch(model, train_loader)
    val_loss, _, _ = evaluate(model, test_loader)
```

```
    scheduler.step(val_loss)

    train_losses.append(train_loss)
    val_losses.append(val_loss)

    print(f"Epoch {epoch+1}: Train {train_loss:.4f} | Val {val_loss:.4f}")
```

```
Epoch 1: Train 0.4173 | Val 0.3504
Epoch 2: Train 0.3559 | Val 0.3131
Epoch 3: Train 0.3334 | Val 0.3181
Epoch 4: Train 0.3221 | Val 0.3320
Epoch 5: Train 0.3166 | Val 0.2871
Epoch 6: Train 0.3044 | Val 0.2821
Epoch 7: Train 0.3011 | Val 0.2779
Epoch 8: Train 0.2911 | Val 0.2766
Epoch 9: Train 0.2880 | Val 0.2820
Epoch 10: Train 0.2815 | Val 0.2822
```

In [127…

```
plt.plot(train_losses, label="Train")
plt.plot(val_losses, label="Validation")
plt.legend()
plt.title("Loss Evolution")
plt.show()
```



In [128…

```
MODEL_PATH = "model/ecg_full_model.pth"

checkpoint = torch.load(MODEL_PATH)

model = ECGNet(n_classes=len(checkpoint["classes"])).to(device)
```

```
model.load_state_dict(checkpoint["model_state"])
model.eval()

class_names = checkpoint["classes"]
thresholds = checkpoint["thresholds"]
```

```
C:\Users\Me\AppData\Local\Temp\ipykernel_8320\1181605888.py:3: FutureWarning: You are using `torch.load` with `weights_only=False` (the current default value), which uses the default pickle module implicitly. It is possible to construct malicious pickle data which will execute arbitrary code during unpickling (See https://github.com/pytorch/pytorch/blob/main/SECURITY.md#untrusted-models for more details). In a future release, the default value for `weights_only` will be flipped to `True`. This limits the functions that could be executed during unpickling. Arbitrary objects will no longer be allowed to be loaded via this mode unless they are explicitly allowlisted by the user via `torch.serialization.add_safe_globals`. We recommend you start setting `weights_only=True` for any use case where you don't have full control of the loaded file. Please open an issue on GitHub for any issues related to this experimental feature.
  checkpoint = torch.load(MODEL_PATH)
```

In [129…

```
from sklearn.preprocessing import MultiLabelBinarizer

mlb = MultiLabelBinarizer()
y_true = mlb.fit_transform(Y["diagnostic_superclass"])

print("y_true shape:", y_true.shape)
print("Classes:", mlb.classes_)
```

```
y_true shape: (21799, 5)
Classes: ['CD' 'HYP' 'MI' 'NORM' 'STTC']
```

In [130…

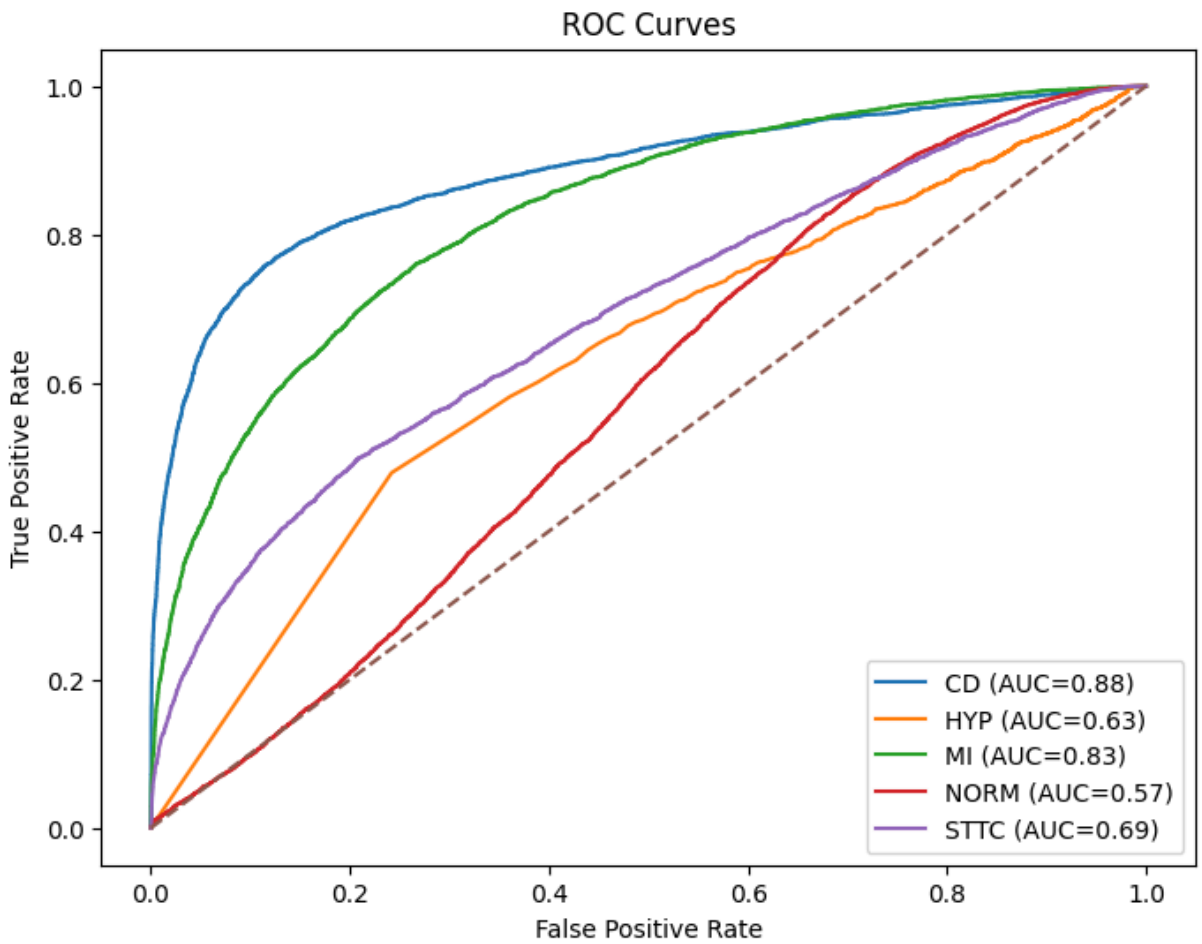
```
probs = []

model.eval()

for signal in X:
    x = torch.tensor(signal, dtype=torch.float32).permute(1, 0).unsqueeze(0).to(dev

    with torch.no_grad():
        logits = model(x)
        p = torch.sigmoid(logits).cpu().numpy()[0]

    probs.append(p)

probs = np.array(probs)

print("probs shape:", probs.shape)
```

```
probs shape: (21799, 5)
```

In [131…

```
from sklearn.metrics import roc_curve, auc

plt.figure(figsize=(8,6))

for i, cls in enumerate(class_names):
    fpr, tpr, _ = roc_curve(y_true[:, i], probs[:, i])
    roc_auc = auc(fpr, tpr)
```

```
    plt.plot(fpr, tpr, label=f"{cls} (AUC={roc_auc:.2f})")

plt.plot([0,1],[0,1],'--')
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curves")
plt.legend()
plt.show()
```



In [132…

```
loss, logits, targets = evaluate(model, test_loader)

probs = torch.sigmoid(logits).cpu().numpy()
y_true = targets.cpu().numpy()

print("y_true:", y_true.shape)
print("probs:", probs.shape)
```

```
y_true: (2198, 5)
probs: (2198, 5)
```

In [133…

```
_, logits, targets = evaluate(model, test_loader)
probs = torch.sigmoid(logits).numpy()
best_thresholds = {}

for i, cls in enumerate(class_names):
    best_f1 = 0
    best_t = 0.5
```

```
    for t in np.arange(0.1, 0.9, 0.05):
        preds = (probs[:, i] >= t).astype(int)
        f1 = f1_score(y_true[:, i], preds)

        if f1 > best_f1:
            best_f1 = f1
            best_t = t

    best_thresholds[cls] = best_t

print("Best thresholds:", best_thresholds)
y_true = targets.numpy()
threshold = 0.3
y_pred = np.zeros_like(probs)

for i, cls in enumerate(class_names):
    t = best_thresholds[cls]
    y_pred[:, i] = (probs[:, i] >= t).astype(int)

torch.save({
    "model_state": model.state_dict(),
    "thresholds": best_thresholds,
    "classes": class_names
}, "ecg_full_model.pth")
```

```
Best thresholds: {'CD': np.float64(0.7000000000000002), 'HYP': np.float64(0.65000000
00000001), 'MI': np.float64(0.45000000000000007), 'NORM': np.float64(0.5500000000000
002), 'STTC': np.float64(0.6500000000000001)}
```

In [134…

```
roc_aucs = {}

for i, cls in enumerate(class_names):
    roc_aucs[cls] = roc_auc_score(y_true[:, i], probs[:, i])

roc_aucs["macro"] = roc_auc_score(y_true, probs, average="macro")

roc_aucs
```

Out[134…

```
{'CD': 0.8971845267427314,
 'HYP': 0.8986716137783106,
 'MI': 0.9274922771403353,
 'NORM': 0.9384363136453644,
 'STTC': 0.9256979090483533,
 'macro': 0.917496528071019}
```

In [135…

```
ap_scores = {}

for i, cls in enumerate(class_names):
    ap_scores[cls] = average_precision_score(y_true[:, i], probs[:, i])

ap_scores["macro"] = average_precision_score(y_true, probs, average="macro")

ap_scores
```

Out[135…

```
{'CD': 0.7991779545930235,
 'HYP': 0.6273219943713791,
 'MI': 0.8341432699783947,
 'NORM': 0.9158381438086864,
 'STTC': 0.802250907505763,
 'macro': 0.7957464540514494}
```

In [136…

```
print(classification_report(
    y_true,
    y_pred,
    target_names=class_names,
    zero_division=0
))
```

```
              precision    recall  f1-score   support

          CD       0.79      0.67      0.73       496
         HYP       0.59      0.59      0.59       262
          MI       0.67      0.84      0.74       550
        NORM       0.81      0.89      0.85       963
        STTC       0.71      0.79      0.75       521

   micro avg       0.74      0.79      0.76      2792
   macro avg       0.71      0.76      0.73      2792
weighted avg       0.74      0.79      0.76      2792
 samples avg       0.75      0.80      0.75      2792
```

In [137…

```
def prepare_input(signal, device):
    x = torch.tensor(signal, dtype=torch.float32)
    x = x.permute(1, 0)
    x = x.unsqueeze(0).to(device)
    return x
```

In [138…

```
print(type(X_test))
print(type(X_test[0]))
print(X_test[0].shape)

def preprocess_signal(signal):
    if isinstance(signal, torch.Tensor):
        signal = signal.cpu().numpy()

    if signal.shape == (1000, 12):
        signal = signal.T

    elif signal.shape == (12, 1000):
        pass

    else:
        raise ValueError(f"Unexpected shape: {signal.shape}")

    return signal


def predict_signal(model, signal, device):
    model.eval()
```

```
    x = prepare_input(signal, device)

    with torch.no_grad():
        logits = model(x)
        probs = torch.sigmoid(logits).cpu().numpy()[0]

    return probs
```

```
<class 'numpy.ndarray'>
<class 'numpy.ndarray'>
(1000, 12)
```

In [139…

```
sample = X_test[0]

probs = predict_signal(model, sample, device)

for cls, p in zip(class_names, probs):
    print(f"{cls}: {p:.3f}")
```

```
CD: 0.100
HYP: 0.012
MI: 0.048
NORM: 0.969
STTC: 0.007
```

In [140…

```
y_pred = np.zeros_like(probs)

for i, cls in enumerate(class_names):
    t = best_thresholds[cls]
    y_pred[i] = int(probs[i] >= t)

for cls, pred in zip(class_names, y_pred):
    print(f"{cls}: {pred}")
```

```
CD: 0.0
HYP: 0.0
MI: 0.0
NORM: 1.0
STTC: 0.0
```

In [141…

```
all_preds = []

for i in range(len(X_test)):
    probs = predict_signal(model, X_test[i], device)

    preds = np.array([
        int(probs[j] >= best_thresholds[class_names[j]])
        for j in range(len(class_names))
    ])

    all_preds.append(preds)

all_preds = np.array(all_preds)
```

In [142…

```
from sklearn.metrics import classification_report
```

```
print(classification_report(y_test, all_preds, target_names=class_names))
```

```
              precision    recall  f1-score   support

          CD       0.79      0.67      0.73       496
         HYP       0.59      0.59      0.59       262
          MI       0.67      0.84      0.74       550
        NORM       0.81      0.89      0.85       963
        STTC       0.71      0.79      0.75       521

   micro avg       0.74      0.79      0.76      2792
   macro avg       0.71      0.76      0.73      2792
weighted avg       0.74      0.79      0.76      2792
 samples avg       0.75      0.80      0.75      2792
```

```
c:\Users\Me\Desktop\ECG\ecg_env\lib\site-packages\sklearn\metrics\_classification.p
y:1731: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in sam
ples with no predicted labels. Use `zero_division` parameter to control this behavio
r.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Me\Desktop\ECG\ecg_env\lib\site-packages\sklearn\metrics\_classification.p
y:1731: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in sample
s with no true labels. Use `zero_division` parameter to control this behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
c:\Users\Me\Desktop\ECG\ecg_env\lib\site-packages\sklearn\metrics\_classification.p
y:1731: UndefinedMetricWarning: F-score is ill-defined and being set to 0.0 in sampl
es with no true nor predicted labels. Use `zero_division` parameter to control this
behavior.
  _warn_prf(average, modifier, f"{metric.capitalize()} is", result.shape[0])
```

In [146…

```
loss, logits, targets = evaluate(model, test_loader)

probs = torch.sigmoid(logits).cpu().numpy()
y_true = targets.cpu().numpy()

print(probs.shape)
```

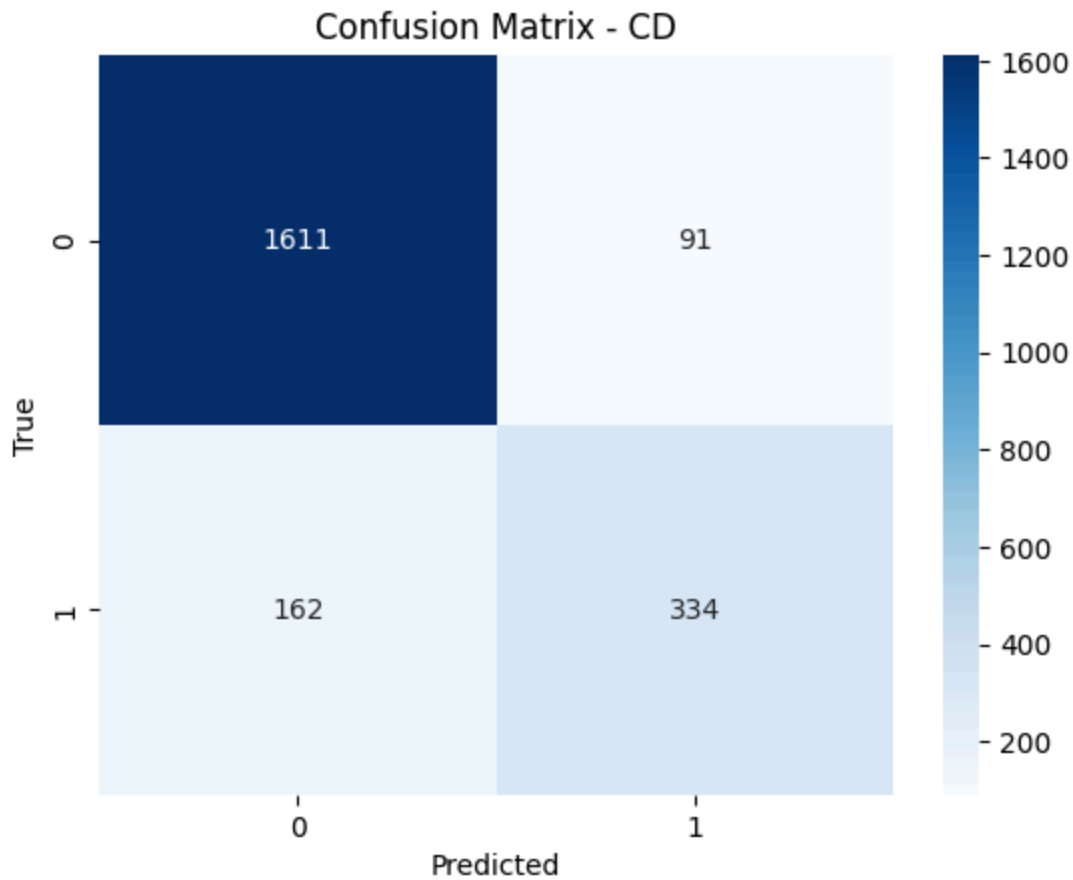
```
(2198, 5)
```

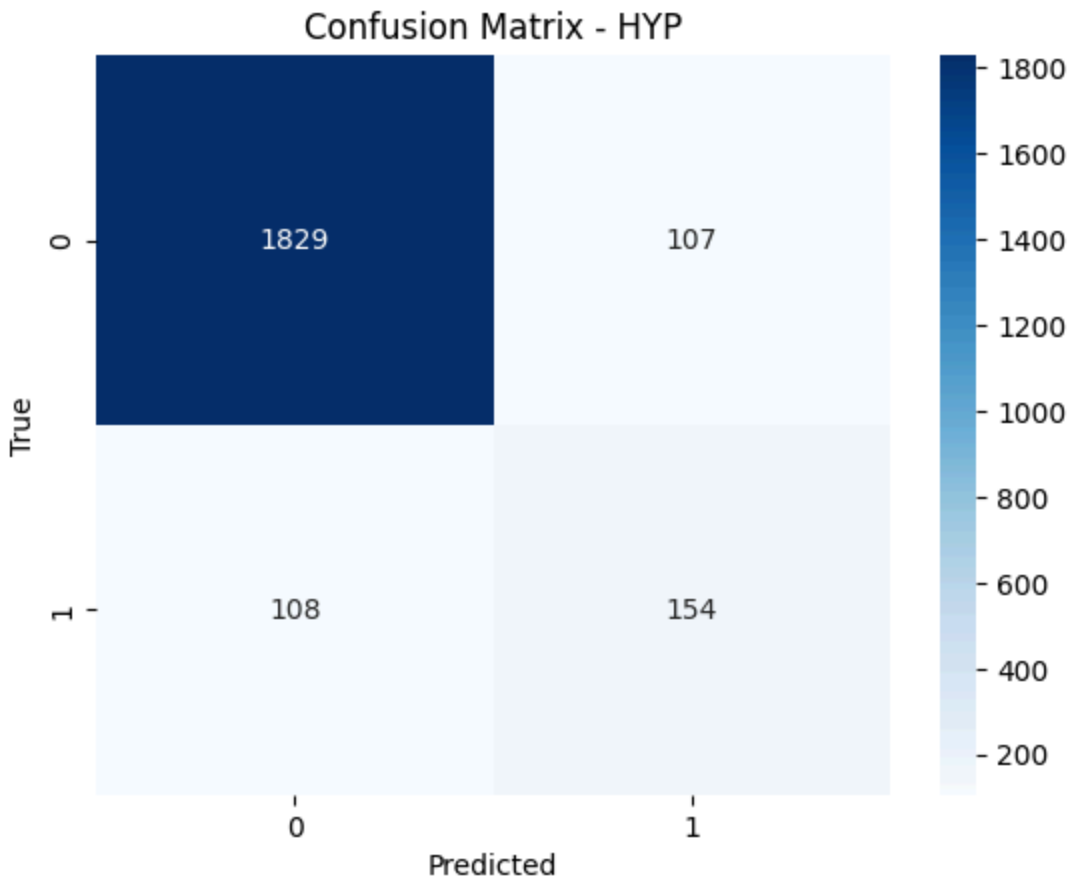
In [147…

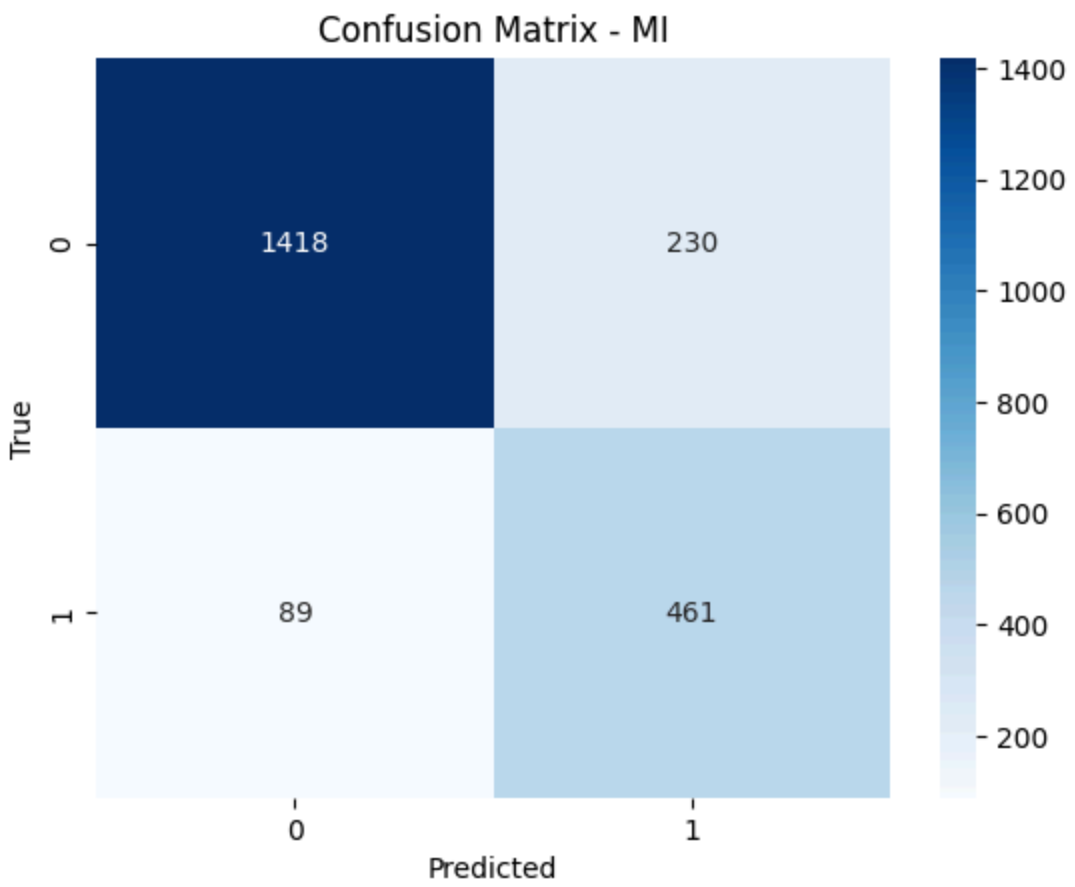
```
from sklearn.metrics import multilabel_confusion_matrix
import seaborn as sns

y_pred = np.zeros_like(probs)

for i, cls in enumerate(class_names):
    t = best_thresholds[cls]
    y_pred[:, i] = (probs[:, i] >= t).astype(int)

print("y_true:", y_true.shape)
print("y_pred:", y_pred.shape)

cm = multilabel_confusion_matrix(y_true, y_pred)

for i, cls in enumerate(class_names):
    plt.figure()
    sns.heatmap(cm[i], annot=True, fmt="d", cmap="Blues")
    plt.title(f"Confusion Matrix - {cls}")
```

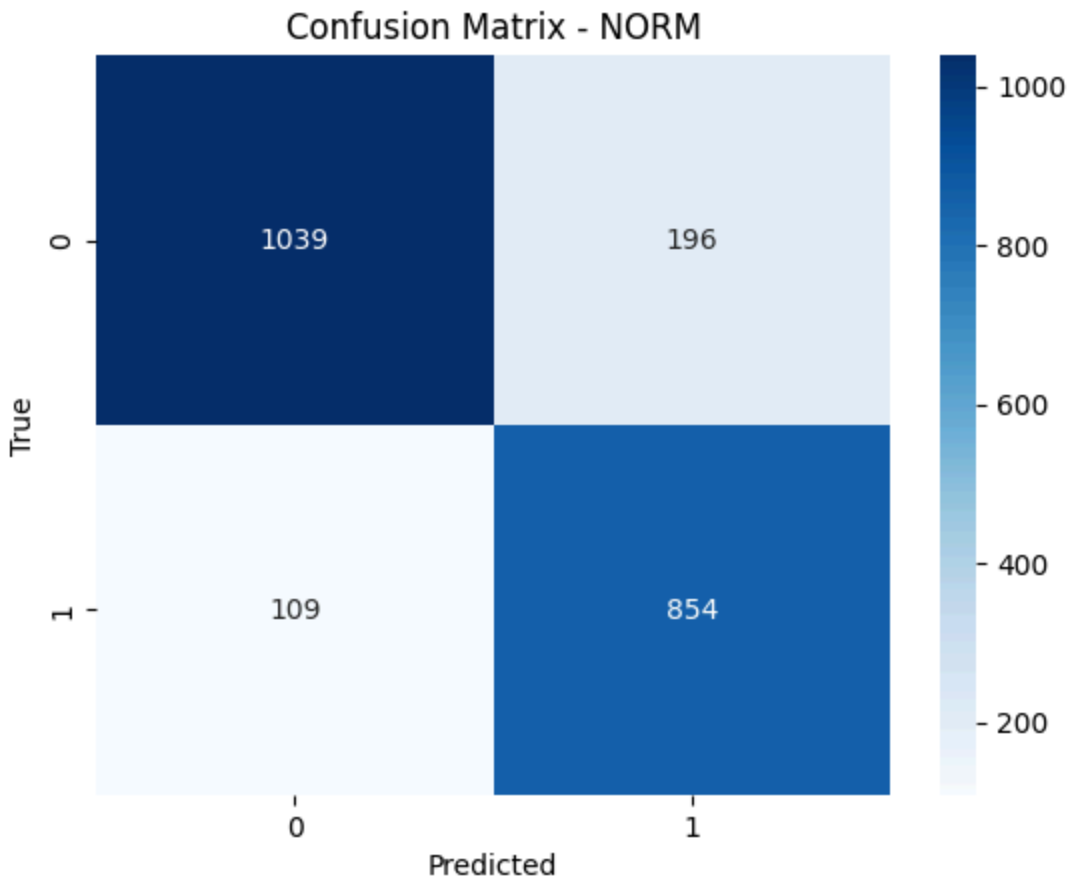
```
    plt.xlabel("Predicted")
    plt.ylabel("True")
    plt.show()
```

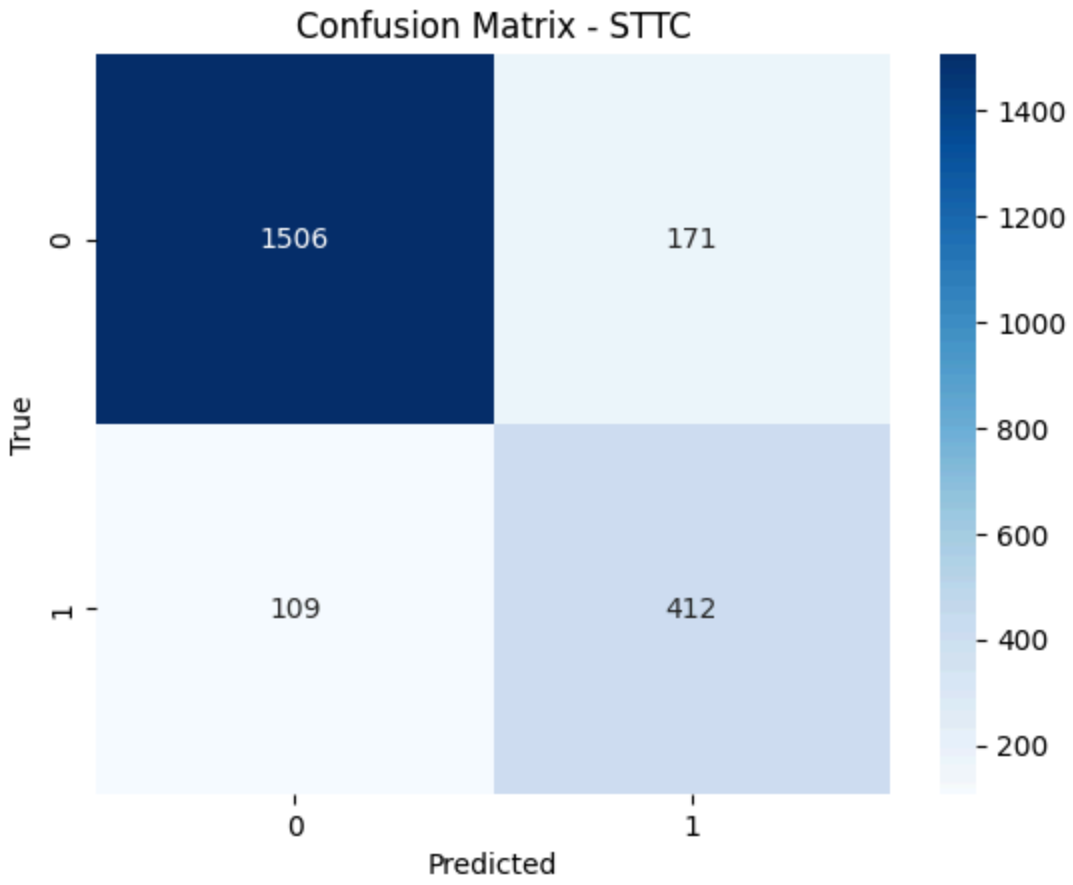
```
y_true: (2198, 5)
y_pred: (2198, 5)
```




Confusion Matrix - HYP
1829
107
108
154
True
Predicted
0
1
1800
1600
1400
1200
1000
800
600
400
200


Confusion Matrix - MI
1418
230
89
461
True
Predicted
0
1
1400
1200
1000
800
600
400
200


Confusion Matrix - NORM
1039
196
109
854
True
0
1
Predicted
0
1
1000
800
600
400
200


Confusion Matrix - STTC
1506
171
109
412
True
0
1
Predicted
0
1
1400
1200
1000
800
600
400
200

In [149…
```
np.save("sample.npy", X_test[0])
```

In [150…
```
print(X_test[0].shape)
```

```
(1000, 12)
```